

Исследование данных базы данных вакансий с hh.ru

Цели и задачи проекта

Наша цель – провести анализ базы данных вакансий с hh.ru, чтобы понять текущие тенденции на рынке труда для аналитиков.

Описание данных

База данных состоит из одной таблицы `public.parcing_table`. Вот её описание:

- `id` — Идентификатор объявления Первичный ключ;
- `name` — Название вакансии;
- `published_at` — Дата выставления вакансии;
- `employer` — Компания, которая ищет работника;
- `department` — Отделение компании, в котором необходим сотрудник;
- `area` — Расположение компании;
- `experience` — Необходимый опыт и ранг кандидата;
- `schedule` — График работы;
- `employment` — Занятость: полная или частичная;
- `salary_from` — Заработная плата в категории «от»;
- `salary_to` — Заработная плата в категории «до»;
- `salary_bin` — Заработная плата в категории «зарплатная вилка»;
- `key_skills_1` — Первый ключевой навык, который требуется от кандидата;
- `key_skills_2` — Второй ключевой навык, который требуется от кандидата;
- `key_skills_3` — Третий ключевой навык, который требуется от кандидата;
- `key_skills_4` — Четвёртый ключевой навык, который требуется от кандидата;
- `soft_skills_1` — Первый дополнительный навык, который требуется от кандидата;
- `soft_skills_2` — Второй дополнительный навык, который требуется от кандидата;
- `soft_skills_3` — Третий дополнительный навык, который требуется от кандидата;
- `soft_skills_4` — Четвёртый дополнительный навык, который требуется от кандидата.

Что нужно сделать

1. Определите диапазон заработных плат в общем, а именно средние значения, минимумы и максимумы нижних и верхних порогов зарплаты.
2. Выявите регионы и компании, в которых сосредоточено наибольшее количество вакансий.
3. Проанализируйте, какие преобладают типы занятости, а также графики работы.
4. Изучите распределение грейдов (Junior, Middle, Senior) среди аналитиков данных и системных аналитиков.

5. Выявите основных работодателей, предлагаемые зарплаты и условия труда для аналитиков.
6. Определите наиболее востребованные навыки (как жёсткие, так и мягкие) для различных грейдов и позиций.

Содержимое проекта

1. Загрузка и знакомство с данными
2. Проверка ошибок в данных и предобработка
3. Ответы на вопросы
4. Итоговый вывод

1. Загрузка данных и знакомство с ними

```
In [1]: # Импортируем библиотеку pandas
import pandas as pd
```

```
In [2]: # Выгружаем данные из датасета в датафрейм df
df = pd.read_csv('parcing_table_202605011654.csv')
```

```
In [3]: df.head()
```

```
Out[3]:
```

	id	name	published_at	employer	department	area	experience	sche
0	100069131	Дата аналитик	2024-05-24 13:05:01.000	СБЕР	Сбер для экспертов	Санкт-Петербург	Junior+ (1-3 years)	Пол
1	100069821	Аналитик данных	2024-06-10 16:49:49.000	МТС	«МТС»	Казань	Junior+ (1-3 years)	Пол
2	100071014	Аналитик данных	2024-06-07 11:08:22.000	Россети Урал	NaN	Екатеринбург	Junior+ (1-3 years)	Пол
3	100077503	Data Analyst	2024-05-24 14:14:00.000	СБЕР	Сбер для экспертов	Москва	Middle (3-6 years)	Пол
4	100077910	Data Analyst / Data Scientist	2024-06-11 14:17:47.000	Итсен	NaN	Москва	Middle (3-6 years)	Пол

```
In [4]: # Выводим информацию о датафрейме
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1801 entries, 0 to 1800
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     1801 non-null   int64
1   name                   1801 non-null   object
2   published_at           1801 non-null   object
3   employer               1801 non-null   object
4   department             521 non-null    object
5   area                   1801 non-null   object
6   experience             1801 non-null   object
7   schedule              1801 non-null   object
```

```

8   employment      1801 non-null    object
9   salary_from      348 non-null    float64
10  salary_to        228 non-null    float64
11  salary_bin       1801 non-null    object
12  key_skills_1     1418 non-null    object
13  key_skills_2     1160 non-null    object
14  key_skills_3     1048 non-null    object
15  key_skills_4     937 non-null    object
16  soft_skills_1    588 non-null    object
17  soft_skills_2    110 non-null    object
18  soft_skills_3     22 non-null    object
19  soft_skills_4     5 non-null     object
dtypes: float64(2), int64(1), object(17)
memory usage: 281.5+ KB

```

Промежуточный вывод

Сделаем промежуточный вывод о полученных данных:

Объём и соответствие описанию: Данные загружены корректно: датафрейм содержит 1801 строку и 20 столбцов. Количество столбцов и их названия соответствуют описанию таблицы `public.parcings_table`.

Пропуски: В данных есть пропуски. Особенно много пропусков в столбцах с зарплатой: `salary_from` заполнен только в 348 строках, а `salary_to` — в 228 строках. Также есть пропуски в `department`, навыках `key_skills_1–4` и `soft_skills_1–4`. Это важно учитывать при предобработке: пропуски в зарплатах нельзя просто удалять без анализа, так как можно потерять большую часть данных.

Типы данных и корректность: `salary_to` — float64, текстовые поля — object. Однако столбец `published_at` сейчас имеет тип object, хотя по смыслу это дата. Его нужно преобразовать в тип datetime.

2. Предобработка данных

Подготовим данные к исследовательскому анализу:

```

In [5]: # Посмотрим какие типы данных есть в каждом столбце
df.dtypes

```

```

Out[5]: id                int64
name                  object
published_at          object
employer              object
department             object
area                  object
experience             object
schedule              object
employment             object
salary_from          float64
salary_to            float64
salary_bin            object
key_skills_1          object
key_skills_2          object
key_skills_3          object
key_skills_4          object
soft_skills_1         object
soft_skills_2         object
soft_skills_3         object

```

```
soft_skills_4      object
dtype: object
```

```
In [6]: # Преобразуем столбец published_at из типа данных object в тип данных datetime
df['published_at'] = pd.to_datetime(df['published_at'])
```

```
In [7]: # Посмотрим какие типы данных есть в каждом столбце
df.dtypes
```

```
Out[7]: id                int64
name                object
published_at        datetime64[ns]
employer            object
department           object
area                object
experience           object
schedule            object
employment           object
salary_from         float64
salary_to           float64
salary_bin          object
key_skills_1         object
key_skills_2         object
key_skills_3         object
key_skills_4         object
soft_skills_1        object
soft_skills_2        object
soft_skills_3        object
soft_skills_4        object
dtype: object
```

```
In [8]: # Выводим количество пропущенных строк в датафрейме
df.isna().sum()
```

```
Out[8]: id                0
name                0
published_at        0
employer            0
department          1280
area                0
experience           0
schedule            0
employment           0
salary_from         1453
salary_to           1573
salary_bin          0
key_skills_1         383
key_skills_2         641
key_skills_3         753
key_skills_4         864
soft_skills_1        1213
soft_skills_2        1691
soft_skills_3        1779
soft_skills_4        1796
dtype: int64
```

```
In [9]: # Подсчитываем процент строк с пропусками
df.isna().sum() / len(df) * 100
```

```
Out[9]: id                0.000000
name                0.000000
published_at        0.000000
employer            0.000000
department          71.071627
```

```
area          0.000000
experience     0.000000
schedule      0.000000
employment    0.000000
salary_from   80.677401
salary_to     87.340366
salary_bin    0.000000
key_skills_1  21.265963
key_skills_2  35.591338
key_skills_3  41.810105
key_skills_4  47.973348
soft_skills_1 67.351471
soft_skills_2 93.892282
soft_skills_3 98.778456
soft_skills_4 99.722376
dtype: float64
```

В `department` пропусков около 71%. Это значит, что отдел указан редко. Можно заменить пропуски на "Не указан":

```
In [10]: df['department'] = df['department'].fillna('Не указан')
```

В `key_skills_1-4` и `soft_skills_1-4` много пропусков, особенно в последних столбцах. Это нормально: не у каждой вакансии указано 4 навыка. Их лучше заменить на "Не указан":

```
In [11]: skill_columns = [
          'key_skills_1', 'key_skills_2', 'key_skills_3', 'key_skills_4',
          'soft_skills_1', 'soft_skills_2', 'soft_skills_3', 'soft_skills_4'
        ]

df[skill_columns] = df[skill_columns].fillna('Не указан')
```

В данных есть пропуски, но они распределены неравномерно. Больше всего пропусков в полях `salary_from`, `salary_to`, `department` и столбцах с навыками. Зарплатные поля не стоит заполнять средними значениями, так как это может исказить анализ. Для анализа зарплат лучше использовать только те вакансии, где зарплата указана. Пропуски в `department` и навыках можно заменить значением "Не указан", так как отсутствие значения здесь означает, что работодатель не предоставил эту информацию.

```
In [12]: # Проверка явных дубликатов по всей строке
df.duplicated().sum()
```

```
Out[12]: 0
```

```
In [13]: # Проверяем неявные дубликаты в датафрейме
df.duplicated(subset='id').sum()
```

```
Out[13]: 0
```

3. Ответы на вопросы

1. Определите диапазон заработных плат в общем, а именно средние значения, минимумы и максимумы нижних и

верхних порогов зарплаты.

```
In [14]: # Диапазон заработных плат: нижний и верхний пороги
salary_stats = df[['salary_from', 'salary_to']].agg(['mean', 'min', 'max']).r
salary_stats
```

```
Out[14]:
```

	salary_from	salary_to
mean	109525.09	153846.71
min	50.00	25000.00
max	398000.00	497500.00

2. Выявите регионы и компании, в которых сосредоточено наибольшее количество вакансий.

```
In [15]: # Топ регионов
top_areas = df['area'].value_counts().head(10)
top_areas
```

```
Out[15]: Москва                1247
Санкт-Петербург             181
Екатеринбург                 51
Новосибирск                  33
Нижний Новгород              33
Владивосток                  31
Казань                       29
Краснодар                    22
Самара                       11
Челябинск                     10
Name: area, dtype: int64
```

```
In [16]: # Топ компаний
top_employers = df['employer'].value_counts().head(10)
top_employers
```

```
Out[16]: СБЕР                243
WILDBERRIES                 43
Ozon                        34
Банк ВТБ (ПАО)              28
T1                           26
МАГНИТ, Розничная сеть     24
МТС                          22
Okko                         19
Центральный банк Российской Федерации  16
Правительство Москвы        15
Name: employer, dtype: int64
```

Наибольшее количество вакансий сосредоточено в крупных городах, таких как Москва, Санкт-Петербург и Екатеринбург. Это объясняется высокой концентрацией бизнеса и IT-компаний в этих регионах. Среди работодателей лидируют компании Сбер, wildberries и Ozon, что может свидетельствовать о высокой потребности этих организаций в аналитиках.

3. Проанализируйте, какие преобладают типы занятости, а также графики работы.

```
In [20]: # Типы занятости
employment_counts = df['employment'].value_counts()
employment_counts
```

```
Out[20]: Полная занятость      1764
Стажировка                   16
Частичная занятость         16
Проектная работа           5
Name: employment, dtype: int64
```

```
In [21]: # Графики работы
schedule_counts = df['schedule'].value_counts()
schedule_counts
```

```
Out[21]: Полный день      1441
Удаленная работа      310
Гибкий график          41
Сменный график         9
Name: schedule, dtype: int64
```

```
In [22]: # Доли типов занятости
employment_percent = (df['employment'].value_counts(normalize=True) * 100).round(2)
employment_percent
```

```
Out[22]: Полная занятость      97.95
Стажировка                   0.89
Частичная занятость         0.89
Проектная работа           0.28
Name: employment, dtype: float64
```

```
In [23]: # Доли графиков работы
schedule_percent = (df['schedule'].value_counts(normalize=True) * 100).round(2)
schedule_percent
```

```
Out[23]: Полный день      80.01
Удаленная работа      17.21
Гибкий график          2.28
Сменный график         0.50
Name: schedule, dtype: float64
```

На рынке вакансий преобладает тип занятости «полная занятость». Это говорит о том, что работодатели чаще ищут аналитиков на постоянную работу, а не на проектную или частичную занятость. Среди графиков работы чаще всего встречается полный день. Также могут встречаться удалённый формат, гибкий график или сменный график, но их доля ниже. Это показывает, что большинство вакансий ориентировано на стандартный рабочий формат.

4. Изучите распределение грейдов (Junior, Middle, Senior) среди аналитиков данных и системных аналитиков.

```
In [28]: # Посмотрим уникальные значения опыта
df['experience'].value_counts()
```

```
Out[28]: Junior+ (1–3 years)      1091
Middle (3–6 years)              555
Junior (no experience)          142
Senior (6+ years)               13
Name: experience, dtype: int64
```

In []:

5. Выявите основных работодателей, предлагаемые зарплаты и условия труда для аналитиков.

In [34]:

```
# 1. Агрегация зарплат и количества вакансий
summary = (
    df.groupby('employer')
      .agg(
          vacancies_count=('id', 'count'),
          salary_from_mean=('salary_from', 'mean'),
          salary_to_mean=('salary_to', 'mean')
      )
      .round(2)
)

# 2. Самые частые значения (mode)
schedule_mode = df.groupby('employer')['schedule'].agg(pd.Series.mode).groupby('employer')
employment_mode = df.groupby('employer')['employment'].agg(pd.Series.mode).groupby('employer')
experience_mode = df.groupby('employer')['experience'].agg(pd.Series.mode).groupby('employer')

# 3. Объединение в одну таблицу
summary['schedule'] = schedule_mode
summary['employment'] = employment_mode
summary['experience'] = experience_mode

# 4. Сортировка и топ-10
summary = summary.sort_values('vacancies_count', ascending=False).head(10)

summary
```

Out [34]:

employer	vacancies_count	salary_from_mean	salary_to_mean	schedule	employment
СБЕР	243	115600.00	101714.29	Полный день	Полная занятость
WILDBERRIES	43	180000.00	220000.00	Полный день	Полная занятость
Ozon	34	NaN	NaN	Полный день	Полная занятость
Банк ВТБ (ПАО)	28	NaN	NaN	Полный день	Полная занятость
T1	26	NaN	NaN	Полный день	Полная занятость
МАГНИТ, Розничная сеть	24	80714.29	81666.67	Удаленная работа	Полная занятость
МТС	22	67500.00	NaN	Полный день	Полная занятость
Okko	19	NaN	NaN	Полный день	Полная занятость
Центральный банк Российской Федерации	16	NaN	NaN	Полный день	Полная занятость

	vacancies_count	salary_from_mean	salary_to_mean	schedule	employment
employer					
Яндекс	15	NaN	NaN	Полный день	Полная занятость

Основными работодателями по количеству вакансий являются СБЕР, WILDBERRIES, Ozon, Банк ВТБ и Т1. Лидером является СБЕР — он разместил 243 вакансии, что значительно больше, чем у остальных компаний. По зарплатам данные представлены не для всех работодателей: у части компаний значения salary_from_mean и salary_to_mean отсутствуют. Среди компаний с указанной зарплатой наиболее высокие средние значения наблюдаются у WILDBERRIES: нижний порог — 180 000 руб., верхний — 220 000 руб. По условиям труда чаще всего встречаются полная занятость и график полный день. Исключением среди топ-компаний является МАГНИТ, где чаще встречается удалённая работа. По опыту большинство вакансий рассчитано на специалистов уровня Junior+ / Middle, чаще всего с опытом от 1 до 3 лет или от 3 до 6 лет.

6. Определите наиболее востребованные навыки (как жёсткие, так и мягкие) для различных грейдов и позиций.

```
In [45]: key_skills = (
    df.melt(
        id_vars='experience',
        value_vars=['key_skills_1', 'key_skills_2', 'key_skills_3', 'key_skills_4'],
        value_name='skill'
    )
    .dropna()
)

key_skills_top = (
    key_skills
    .groupby(['experience', 'skill'])
    .size()
    .reset_index(name='count')
    .sort_values(['experience', 'count'], ascending=[True, False])
    .groupby('experience')
    .head(5)
)

key_skills_top = key_skills_top.reset_index(drop=True)
key_skills_top
```

```
Out [45]:
```

	experience	skill	count
0	Junior (no experince)	Не указан	226
1	Junior (no experince)	SQL	37
2	Junior (no experince)	Python	32
3	Junior (no experince)	Анализ данных	31
4	Junior (no experince)	Коммуникация	17
5	Junior+ (1-3 years)	Не указан	1647
6	Junior+ (1-3 years)	SQL	467

	experience	skill	count
7	Junior+ (1-3 years)	Анализ данных	252
8	Junior+ (1-3 years)	Python	238
9	Junior+ (1-3 years)	Документация	120
10	Middle (3-6 years)	Не указан	759
11	Middle (3-6 years)	SQL	247
12	Middle (3-6 years)	Python	176
13	Middle (3-6 years)	Анализ данных	111
14	Middle (3-6 years)	Документация	97
15	Senior (6+ years)	SQL	9
16	Senior (6+ years)	Не указан	9
17	Senior (6+ years)	Python	6
18	Senior (6+ years)	A/B тесты	3
19	Senior (6+ years)	Документация	3

In [49]:

```
soft_skills = (
    df.melt(
        id_vars='experience',
        value_vars=['soft_skills_1', 'soft_skills_2', 'soft_skills_3', 'soft_skills_4'],
        value_name='skill'
    )
    .dropna()
)

soft_skills_top = (
    soft_skills
    .groupby(['experience', 'skill'])
    .size()
    .reset_index(name='count')
    .sort_values(['experience', 'count'], ascending=[True, False])
    .groupby('experience')
    .head(5)
)

soft_skills_top = soft_skills_top.reset_index(drop=True)
soft_skills_top
```

Out [49]:

	experience	skill	count
0	Junior (no experience)	Не указан	507
1	Junior (no experience)	Коммуникация	17
2	Junior (no experience)	Документация	15
3	Junior (no experience)	Аналитическое мышление	9
4	Junior (no experience)	Аналитическое мышление	6
5	Junior+ (1-3 years)	Не указан	3940
6	Junior+ (1-3 years)	Документация	123
7	Junior+ (1-3 years)	Коммуникация	105
8	Junior+ (1-3 years)	Аналитическое мышление	89

	experience	skill	count
9	Junior+ (1-3 years)	Документация	27
10	Middle (3-6 years)	Не указан	1998
11	Middle (3-6 years)	Документация	93
12	Middle (3-6 years)	Коммуникация	56
13	Middle (3-6 years)	Документация	16
14	Middle (3-6 years)	Аналитическое мышление	11
15	Senior (6+ years)	Не указан	34
16	Senior (6+ years)	Презентация результатов	3
17	Senior (6+ years)	Документация	3
18	Senior (6+ years)	Коммуникация	3
19	Senior (6+ years)	Проактивность	3

Наиболее востребованными жёсткими навыками для аналитиков являются SQL, Python и анализ данных. Эти навыки встречаются на всех уровнях: от Junior до Senior, что говорит об их базовой важности для профессии аналитика. Для уровня Junior чаще всего требуются SQL, Python и анализ данных. Для Junior+ и Middle набор навыков похожий, но количество вакансий значительно выше: особенно часто встречаются SQL, Python, анализ данных и документация. Для Senior также лидируют SQL и Python, но выборка по этому грейду небольшая, поэтому выводы по нему нужно делать осторожно. Среди мягких навыков чаще всего встречаются документация, коммуникация и аналитическое мышление. Для более опытных специалистов также встречаются навыки презентации результатов и проактивность. Важно отметить, что во многих вакансиях навыки не указаны, поэтому значение «Не указан» занимает первое место почти во всех грейдах. Поэтому при интерпретации лучше смотреть на реальные навыки после исключения этого значения.

4. Итоговый вывод

Анализ вакансий с hh.ru показал, что рынок аналитиков в основном ориентирован на специалистов с опытом от 1 до 3 лет и от 3 до 6 лет, то есть на уровне Junior+/Middle и Middle. Вакансий для специалистов без опыта и Senior меньше. Больше всего вакансий сосредоточено в крупных компаниях, таких как СБЕР, WILDBERRIES, Ozon, ВТБ и Т1. Лидером по количеству вакансий является СБЕР. По условиям труда чаще всего предлагаются полная занятость и график полный день. Также встречается удалённая работа, но она менее распространена. Зарплаты указаны не во всех вакансиях, поэтому выводы по ним нужно делать осторожно. Среди компаний с указанной зарплатой высокие значения показал WILDBERRIES: в среднем от 180 000 до 220 000 рублей. Самыми востребованными жёсткими навыками являются SQL, Python и анализ данных. Среди мягких навыков чаще встречаются коммуникация, документация и аналитическое мышление. В целом можно сделать вывод, что работодатели чаще ищут аналитиков с базовым или средним опытом, готовых работать на полной занятости, владеющих SQL, Python и навыками анализа данных.